

AI Career Companion Agent for Internship Matching and Interview Preparation

Document Type: Agile Project Documentation

Project Title: AI Career Companion Agent For Internship Matching And Interview Preparation-INTERNSPHERE

Prepared By: Botsa SriHarshitha

Project Duration: July 22 -September 22

Batch : 02

Team Code:#M-2-3

Mentor: VIJAY PARMAR

Table of Contents

No.	Section	Page No.
1	Project Overview	3
2	Project Objective	4
3	Technology Stack	5
4	Project Requirements	6
5	System Architecture / Workflow	7-8
6	Product Backlog	9
7	Sprint Planning	10
8	Sprint Tasks & Status	12
9	Implementation / Key Features	13-19
10	Testing	20-21
11	Challenges & Solutions	22
12	Database Schema	23
13	Final Project Status	24

1. Project Overview

1.1 Project Name

InternSphere — AI Career Companion Agent for Internship Matching and Interview Preparation

1.2 What is the Project?

InternSphere is a AI-augmented career acceleration and internship matching platform designed to streamline every phase of an intern's application lifecycle. The platform ingests candidate resumes (.pdf and .docx), runs a hybrid deterministic regex and Google Gemini extraction pipeline to build a structured profile, performs explainable semantic and skill-based internship matching over a curated vector index, conducts granular skill-gap analyses, drafts hallucination-free cover letters downloadable as PDFs, answers platform questions through a grounded AI Assistant, and provides an interactive, multi-turn AI Interview Preparation Agent equipped with role recommendations, roadmap synthesis, and browser-based voice read-aloud playback.

1.3 Problem Statement

Internship seekers, college students, and entry-level graduates face substantial challenges when seeking practical industry experience:

1. **Inefficient Internship Discovery:** Generic search filters fail to grasp candidate projects, coursework, and technical competencies, leading to poor role alignment.
2. **Opaque Skill Deficiencies:** Candidates rarely receive actionable feedback regarding why they were rejected or what skills they lack for specific listings.
3. **Time-Consuming Application Customization:** Tailoring unique cover letters and responses for multiple employers consumes significant effort and often leads to generic submissions.
4. **Disjointed Interview Preparation:** Traditional interview study relies on static question lists without contextual adaptation to a candidate's actual projects or technical background.

1.4 Target Users

- **Students:** Undergraduate and graduate students actively seeking summer or semester internships.
- **Internship Seekers & Fresh Graduates:** Entry-level professionals looking to transition academic skills into corporate roles.
- **Career Changers:** Candidates who require personalized skill-gap identification and targeted learning roadmaps to break into tech domains.

2. Project Objective

The primary objective of InternSphere is to provide an end-to-end, intelligent career companion that bridges the gap between candidate qualifications and industry internship openings.

Objectives:

- **Automate Resume Parsing & Information Extraction:** Extracting contact details, education, technical skills, soft skills, and projects from PDF and Word resumes using hybrid regex and LLM parsing.
- **Deliver Explainable Semantic Internship Matching:** Rank internship postings against candidate qualifications using a hybrid scoring algorithm combining 384-dimensional vector similarity and explicit required/preferred skill overlap ratios.
- **Provide Clear Skill-Gap Visibility:** Highlight missing required skills, preferred qualifications, and eligibility prerequisites directly on internship recommendation cards.
- **Automate Tailored Cover Letter Synthesis:** Generate personalized, factual cover letters based strictly on candidate resume data and job facts, complete with in-browser editing and instant binary PDF generation.
- **Deliver Domain-Separated AI Career Coaching:** Provide a dedicated AI Assistant for platform operations and an AI Interview Preparation Agent for mock interviews, STAR-method guidance, and voice playback.

3. Technology Stack

InternSphere is engineered with lightweight, high-performance, and verifiable technologies:

Category	Technology	Purpose / Justification
Programming Language	Python 3.12+	Core backend language providing native async performance and typing.
Backend Framework	FastAPI (v0.115+)	High-performance ASGI framework for RESTful APIs and static file routing.
ASGI Server	Uvicorn (v0.30+)	Production-ready ASGI server running the FastAPI backend.
Frontend UI	Vanilla HTML5 & CSS3	Custom responsive design system with dark/light theme support and glassmorphism.
Frontend Logic	Vanilla JavaScript (ES6+)	Modular asynchronous client-side state management without heavy frameworks.
Voice / Browser APIs	Web Speech API	Native window.speechSynthesis for interview prep audio narration.
Database	SQLite / PostgreSQL	Local relational development on resume_parser.db; production-ready with PostgreSQL.
Database ORM	SQLAlchemy 2.0+	Declarative ORM models, relational cascades, and safe SQLite schema migrations.
Authentication	JWT (python-jose) & bcrypt	Stateless JSON Web Tokens (HS256) and secure password hashing (passlib).
AI / LLM Engine	Google Gemini API (google-gemai)	Powered by gemini-3.5-flash-lite, gemini-3.6-flash, and gemini-flash-latest.
Vector Store / RAG	Persisted JSON Vector Index	Lightweight 384-dimensional cosine similarity store (data/internship_vector_index.json).
Embeddings	Deterministic Feature Hashing / Gemini	Fast local SHA-256 word n-gram feature hashing (offline) or gemini-embedding-001.
Document Processing	pdfplumber & python-docx	PDF text extraction and DOCX document stream parsing.
PDF Generation	Custom Python PDF-1.4 Generator	Lightweight, zero-dependency binary PDF builder for cover letters.
Testing	Python unittest	Automated unit and integration test suite across routers and service logic.
Version Control	Git & GitHub	Distributed source code management and collaborative version control.

4. Project Requirements

Functional Requirements

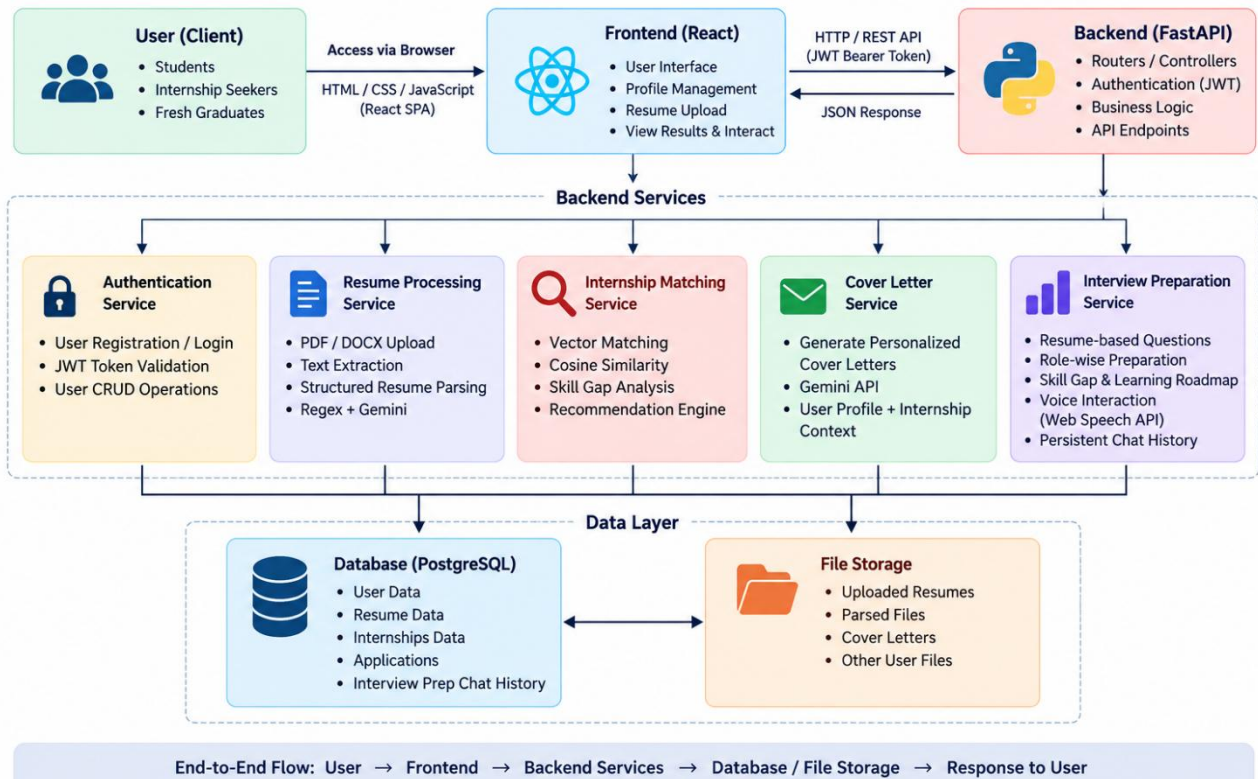
- **User Authentication & Authorization:** Users can register, log in, change passwords, and request password reset tokens; endpoints are secured with JWT bearer tokens.
- **Profile Management:** Users can update personal details, college, GPA, social links, technical skills, and upload profile pictures.
- **Resume Upload & Parsing:** Users can upload .pdf or .docx resumes (up to 5 MB); the system parses and stores structured candidate facts.
- **Semantic Internship Retrieval & Matching:** System matches candidate resumes against curated internships and displays match scores with skill gap breakdowns.
- **Application Tracking:** Candidates can submit, view, update, and withdraw applications with prevention of duplicate entries.
- **Cover Letter Generation & Export:** System drafts role-tailored cover letters using candidate facts, allows in-browser edits, and exports to TXT and PDF.
- **Grounded AI Platform Assistant:** Multi-turn chat assistant providing platform navigation and FAQ answers backed by documentation RAG.
- **AI Interview Preparation Coach:** Generates Technical, HR, and Project questions, provides STAR answer advice, produces learning roadmaps, and speaks answers aloud.
- **Custom Study Document RAG:** Users can upload domain-specific study notes or job descriptions for grounded interview prep.

Non-Functional Requirements

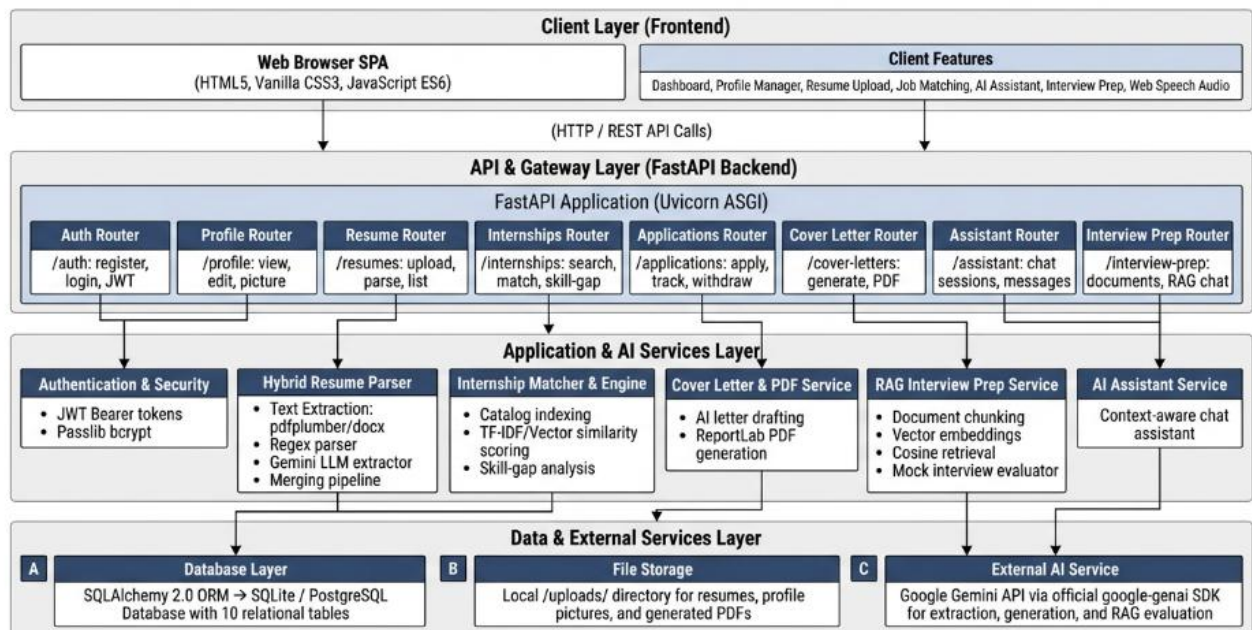
- **Security:** Passwords encrypted with bcrypt; JWT access tokens expire in 60 minutes; owner-only access controls enforced across all personal data.
- **Performance & Latency:** Sub-second deterministic local vector search; asynchronous background LLM invocations.
- **Reliability & Anti-Hallucination:** Prompts enforce fact-bounded generation for cover letters and assistant answers.
- **Usability:** Responsive single-page interface with real-time toast feedback, modal confirmation dialogs, and dark/light themes.
- **Maintainability:** Modular architecture isolating database models, API routers, and business logic services.

5. System Architecture / Workflow

High-Level System Architecture Diagram



System Architecture Diagram



Core Workflows

1. Resume Ingestion & Parsing Workflow

Candidate Upload (.pdf/.docx) → Text Extraction → [Regex Parser: Contacts, Links] + [Gemini LLM: Skills, Projects, Experience] → merge.py → resumes DB Table

2. RAG & Internship Matching Workflow

Candidate Facts → Query Vector (384-d) → Cosine Search on Index → Top-K Listings → Skill Gap Analysis → Ranked Cards

3. AI Assistant Workflow

User Query → Session Context Retrieval → RAG over product_knowledge.json → Gemini Prompt → DB Persistence → Answer Display

4. Interview Preparation Workflow

Parsed Resume → Target Role Selection → Gemini Q&A / Roadmap Engine → STAR Guidance → Web Speech Read-Aloud

6. Product Backlog

ID	Feature / Task	Module	Priority	Status
PB-01	Project Initialization & FastAPI Static SPA Setup	Core / Infra	High	Done
PB-02	Database Configuration (SQLAlchemy, SQLite, Safe Schema Migrations)	Database	High	Done
PB-03	User Authentication (Registration, Login, JWT Token Issuance, Password Reset)	Auth	High	Done
PB-04	User Profile Management (Bio, Skills, Education, GPA, Picture Upload)	Profile	High	Done
PB-05	Hybrid Resume Extraction Pipeline (pdfplumber, python-docx, Regex, Gemini)	Resume	High	Done
PB-06	Vector Indexing & Ingestion Engine (internship_vector_index.json)	RAG / Search	High	Done
PB-07	Explainable Internship Matching Algorithm & Skill-Gap Breakdown	Matching	High	Done
PB-08	Application Lifecycle Management (Apply, Track, Update Status, Withdraw)	Applications	Medium	Done
PB-09	AI Cover Letter Generator with Fact-Bounded Prompts & PDF-1.4 Export	Cover Letter	Medium	Done
PB-10	Grounded Product AI Assistant with Persistent Multi-Turn Chat Sessions	AI Assistant	Medium	Done
PB-11	AI Interview Preparation Agent (Role Recommendations, Questions, Roadmaps)	Interview Prep	High	Done
PB-12	Custom Study Document Upload & Retrieval for Interview Grounding	Interview Prep	Medium	Done
PB-13	Web Speech API Audio Synthesis Integration (window.speechSynthesis)	Frontend / UI	Medium	Done
PB-14	Comprehensive Automated Test Suite (unittest Discovery)	Testing	High	Done
PB-15	Production Cloud Deployment & Containerization Setup	DevOps	Low	To Do

7. Sprint Planning

The project was executed following an Agile Scrum development framework organized into five distinct 1-week sprints:

Sprint 1: Foundation, Architecture & Authentication

Duration: *1 week*

Goal: Initialize application architecture, establish database engine, and implement secure user authentication.

Sprint Backlog:

- Establish directory structure and virtual environment.
- Configure SQLAlchemy with automatic table creation and SQLite foreign key support.
- Implement /register, /login, /me, and password reset APIs.
- Build SPA landing layout with registration and login forms.

Sprint 2: Resume Ingestion & Profile Management

Duration: *one and half week*

Goal: Deliver owner-managed profiles and the hybrid resume parsing engine.

Sprint Backlog:

- Implement /profile CRUD endpoints and profile picture uploads.
- Integrate pdfplumber and python-docx for raw document text extraction.
- Build regex_extractor.py for deterministic contact/link parsing.
- Connect llm_extractor.py for structured entity extraction via Gemini.
- Combine extracted data via merge.py and persist into the resumes table.

Sprint 3: Semantic Internship Matching & Applications

Duration: *one and half week*

Goal: Implement vector embeddings, semantic RAG matching, and application tracking.

Sprint Backlog:

- Construct 384-dimensional feature hashing and Gemini embedding indexers.
- Build the matching algorithm combining cosine similarity (35%) and skill ratios (65%).
- Implement skill-gap calculations (matching, missing required, missing preferred).
- Create the application lifecycle module (/applications) with duplicate prevention.
- Design interactive matching cards and modal detail dialogs.

Sprint 4: AI Career Services(AI ASSISTANT), Cover Letters

Duration: *two weeks*

Goal: Develop AI-assisted career tools including cover letters, platform assistant, and interview preparation.

Sprint Backlog:

- Build fact-bounded Gemini cover letter generation with custom PDF-1.4 downloads.
- Develop the Product AI Assistant using RAG over data/product_knowledge.json.
- Implement the Interview Preparation Agent with role recommendations and roadmaps.

Sprint 5: Interview Preparation Agent ,Testing, Integration, Hardening & Finalization

Duration: *two weeks*

Goal: Perform comprehensive testing, resolve edge cases, refine UI responsiveness, and document the project.

Sprint Backlog:

- Integrate browser Web Speech API for voice playback of coaching responses.
- Add session persistence (interview_sessions) to retain multi-turn context.
- Write test suites for candidate matching, session isolation, and interview APIs.
- Implement safe schema migration utilities (apply_safe_schema_updates).
- Sanitize Gemini error responses to eliminate credential leak risks.
- Complete technical documentation and GitHub repository documentation.

8. Sprint Tasks & Status

Sprint	Task Description	Status
Sprint 1	Project setup, folder conventions, and .env handling	Done
Sprint 1	SQLAlchemy engine & relational model definitions (models.py)	Done
Sprint 1	JWT token generation, bcrypt hashing, and auth endpoints	Done
Sprint 1	Base single-page frontend structure (index.html, styles.css)	Done
Sprint 2	Profile management endpoints & image upload pipeline	Done
Sprint 2	PDF & DOCX text extraction pipeline (extract_text.py)	Done
Sprint 2	Regex and Gemini LLM structured parsing integration	Done
Sprint 2	Profile & Resume management dashboard UI	Done
Sprint 3	Persisted JSON vector indexer (internship_index.py)	Done
Sprint 3	Hybrid scoring algorithm & skill-gap analysis logic	Done
Sprint 3	Application tracking APIs & status withdrawal endpoints	Done
Sprint 3	Interactive matching dashboard with skill tags and filters	Done
Sprint 4	Gemini cover letter generator & pure-Python PDF builder	Done
Sprint 4	Product AI Assistant with RAG knowledge retrieval	Done
Sprint 4	Interview Preparation Agent with tailored question generation	Done
Sprint 4	Multi-turn chat persistence in database tables	Done
Sprint 4	Web Speech API audio synthesis (Audio Listen controls)	Done
Sprint 5	Automated test suite execution across all test files	Done
Sprint 5	Out-of-scope question routing & credential sanitization	Done
Sprint 5	Complete documentation, Agile report, and PPT structure	Done

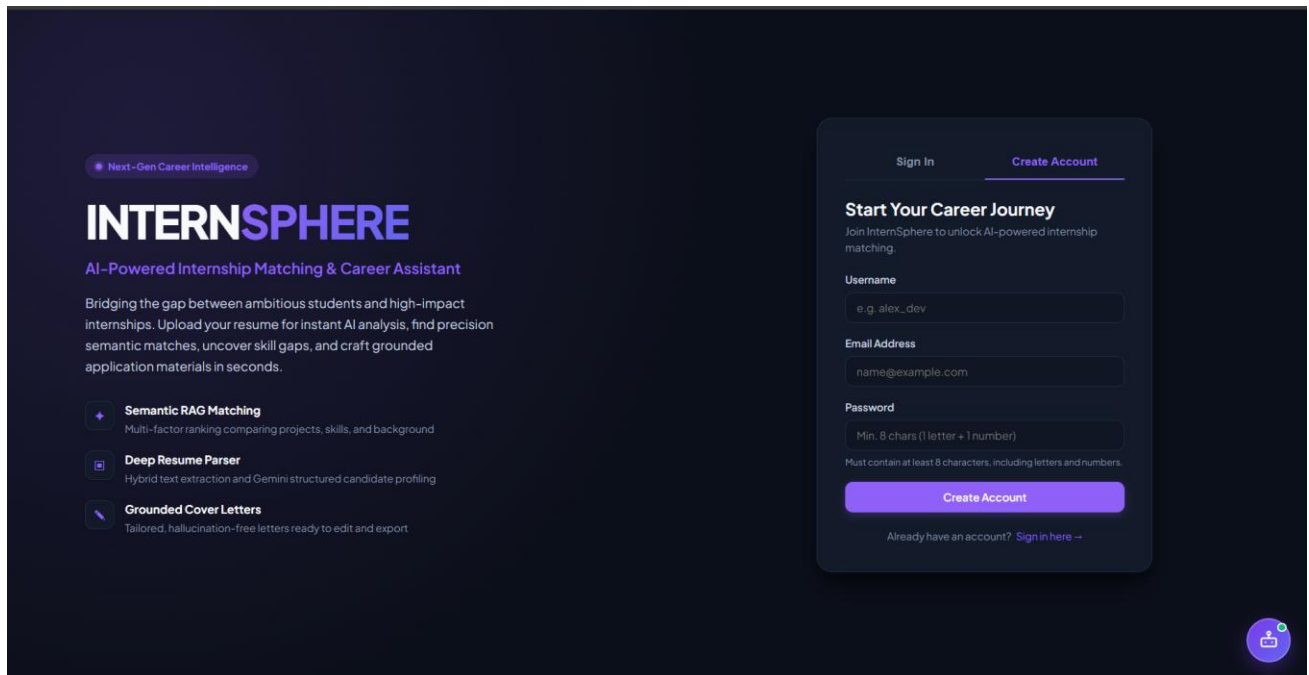
9. Implementation / Key Features

Feature 1: User Authentication & Profile Management

Description: Secure account registration, authentication, password management, and personal profile administration.

Implementation: FastAPI routers (auth_routes.py, profile_routes.py) backed by passlib[bcrypt] and python-jose. User profiles support detailed education history, branch, graduation year, CGPA, and image uploads.

Output / Result:

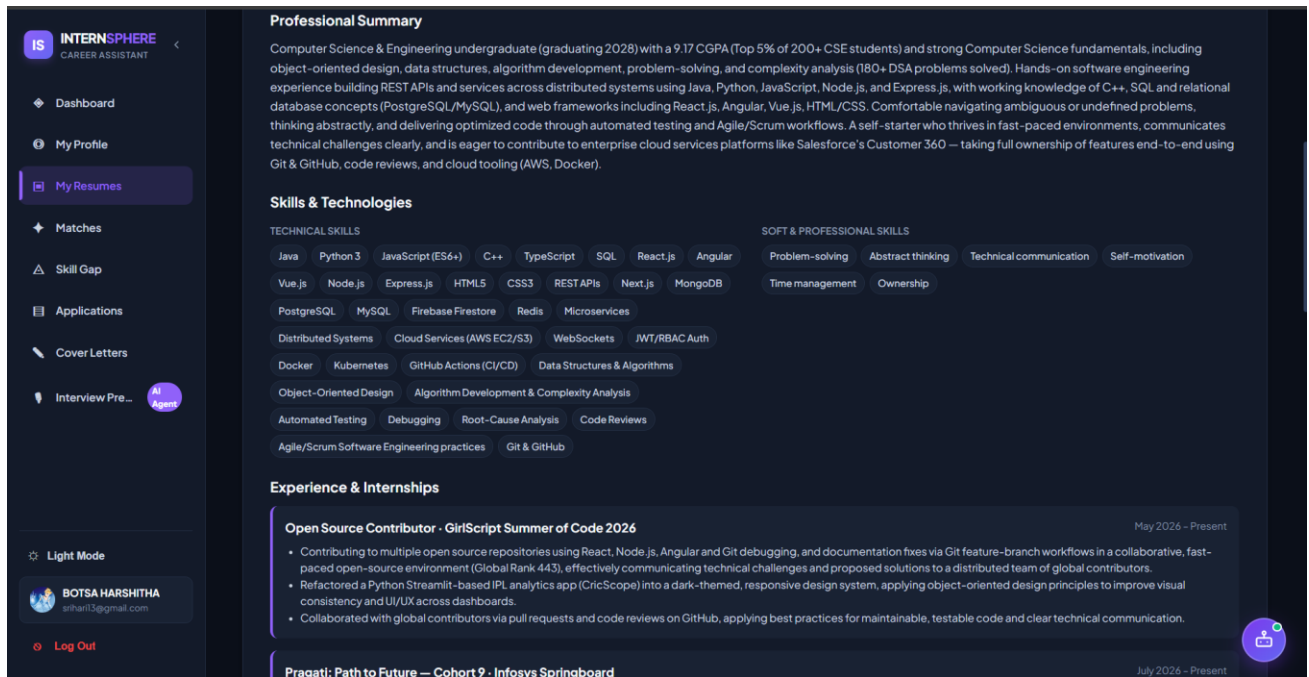
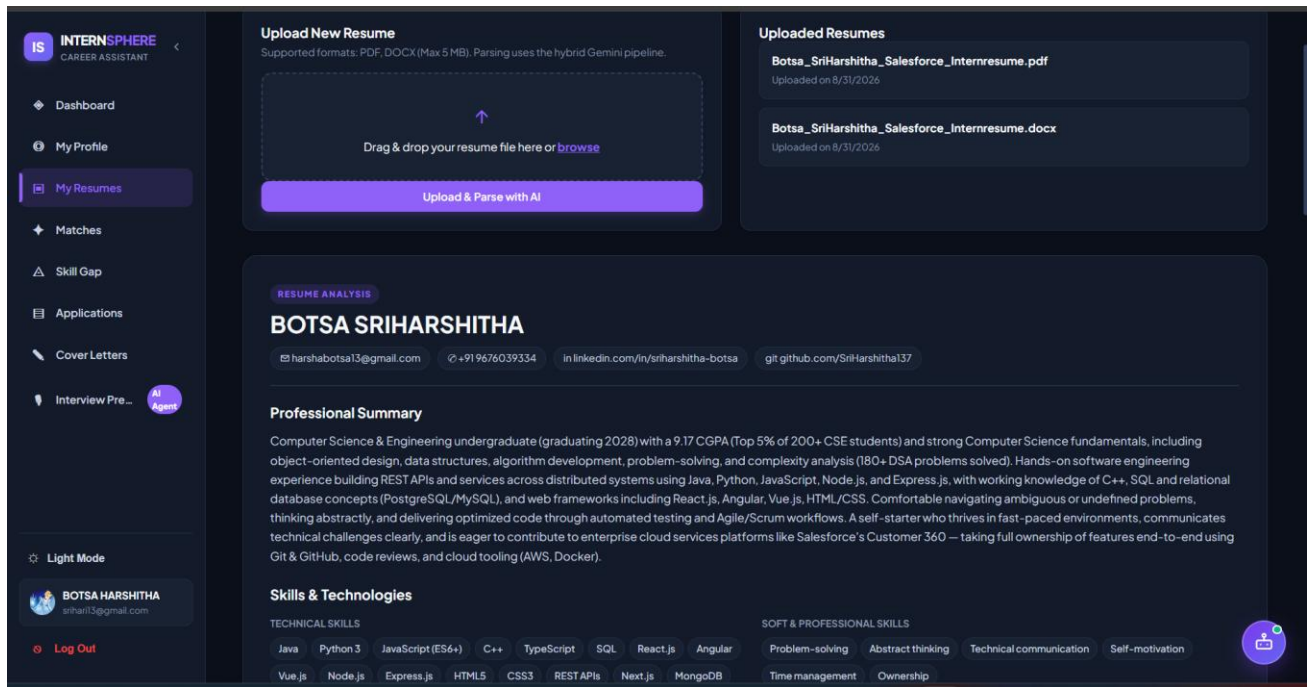


Feature 2: Hybrid Resume Parsing Pipeline

Description: Automatic parsing of candidate resumes in .pdf or .docx format into structured JSON.

Implementation: Text is extracted via pdfplumber or python-docx. Deterministic regular expressions parse contact details and URLs. Google Gemini extracts structured skill lists, employment history, and education. Results are unified and stored in SQLite.

Output / Result:

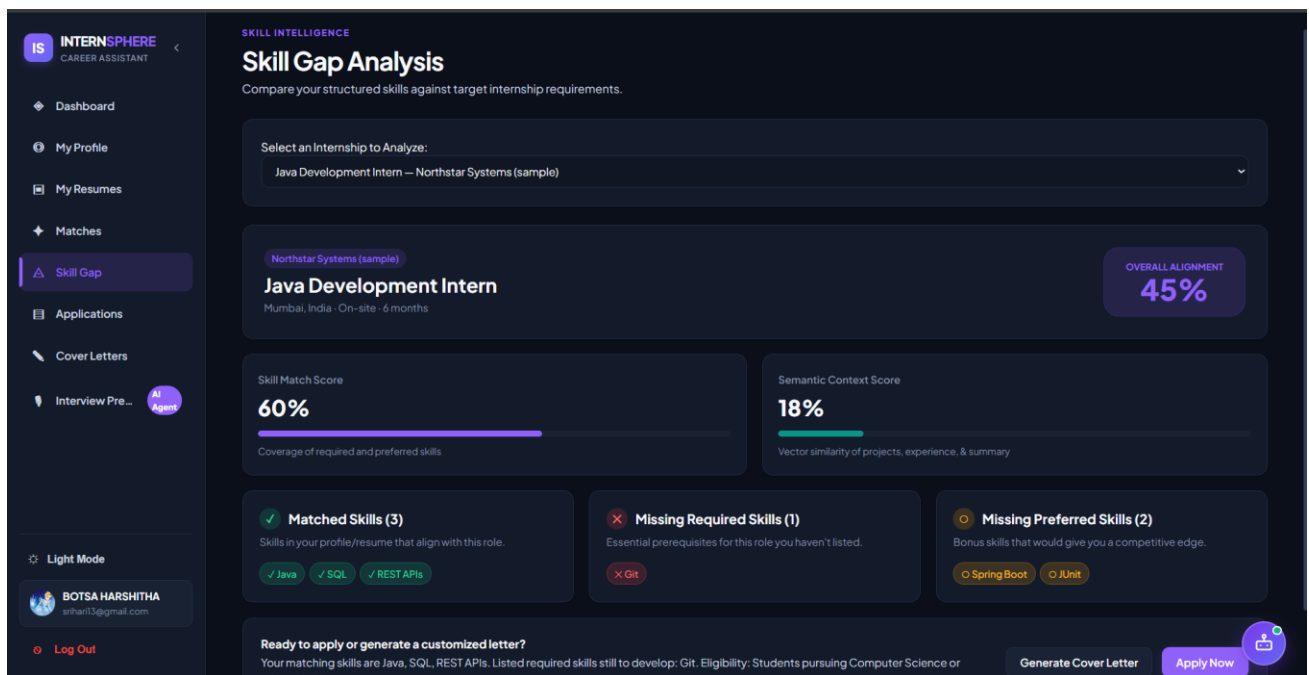
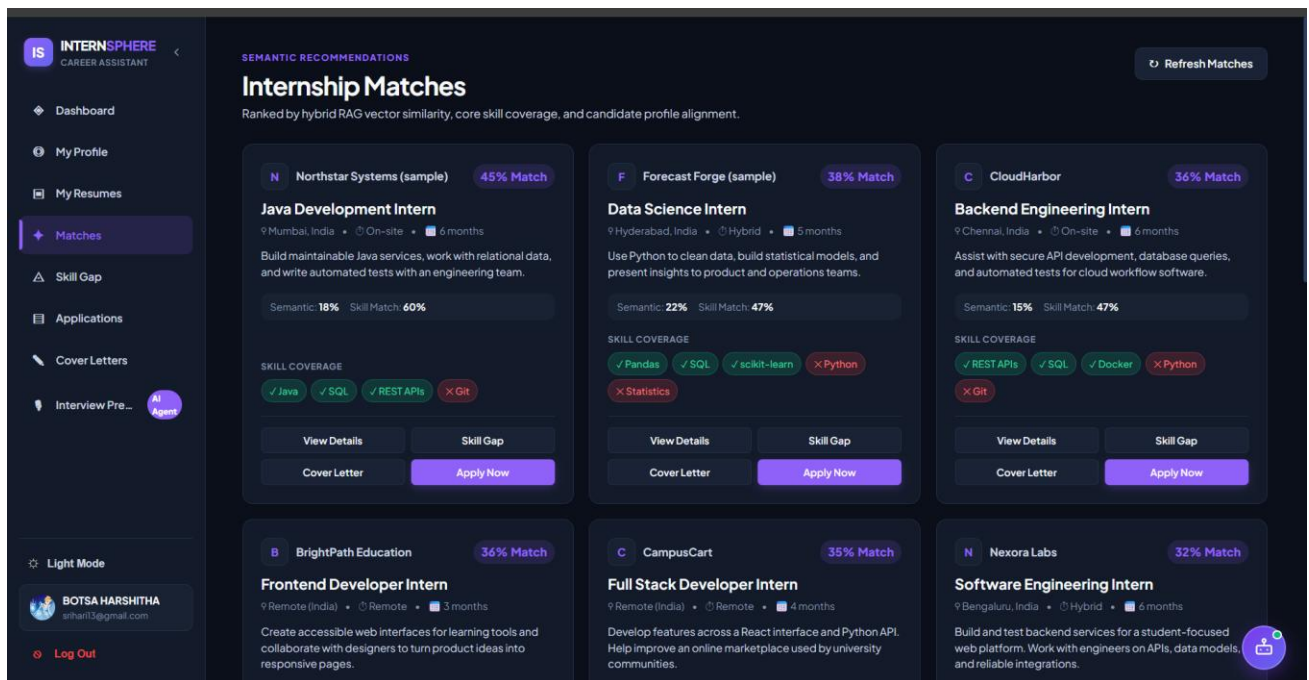


Feature 3: Semantic Internship Matching & Skill-Gap Analysis

Description: Intelligent recommendation engine matching candidate resumes against internships, providing match percentages and skill gap breakdowns.

Implementation: Candidate facts are embedded into a 384-dimensional vector and compared against internship listings using cosine similarity. A weighted scoring function (35% semantic + 65% skill ratio) determines rankings. Missing required and preferred skills are clearly itemized.

Output / Result:

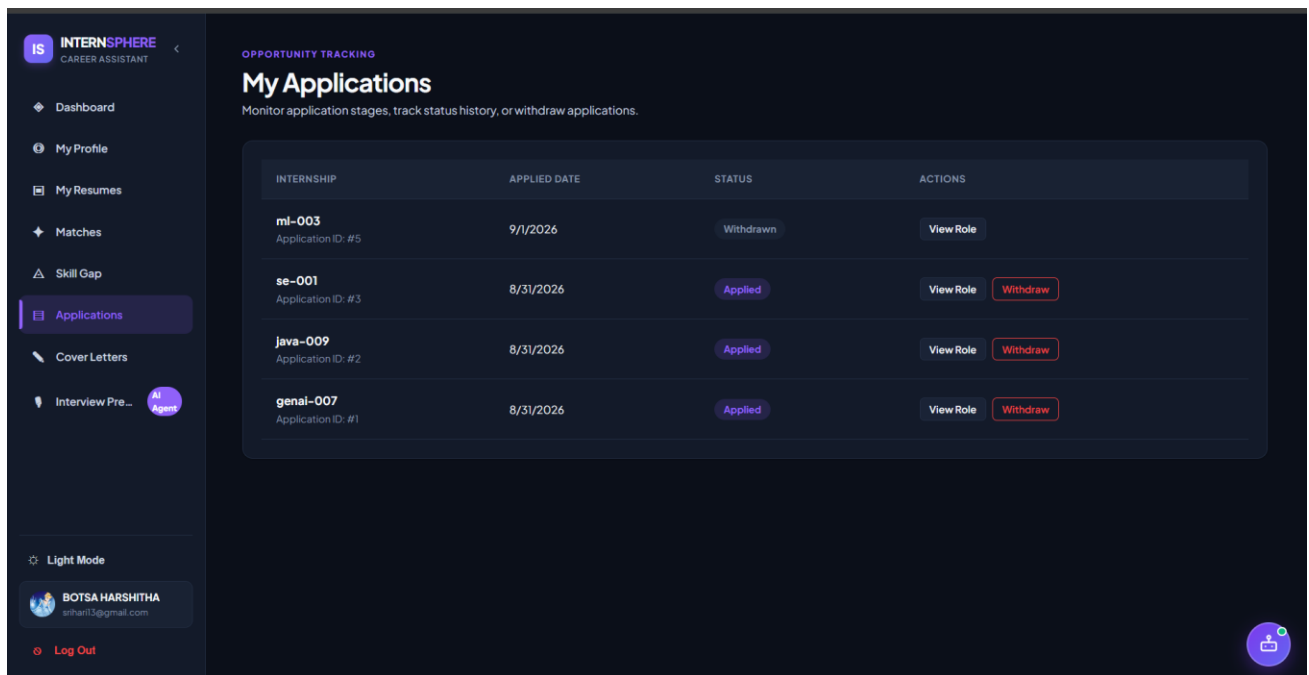


Feature 4: Application Lifecycle Tracking

Description: Comprehensive application tracking allowing students to apply, review application statuses, and withdraw active submissions.

Implementation: The applications table links users to specific internships with attached resumes and cover letters. Unique constraints prevent duplicate submissions.

Output / Result:

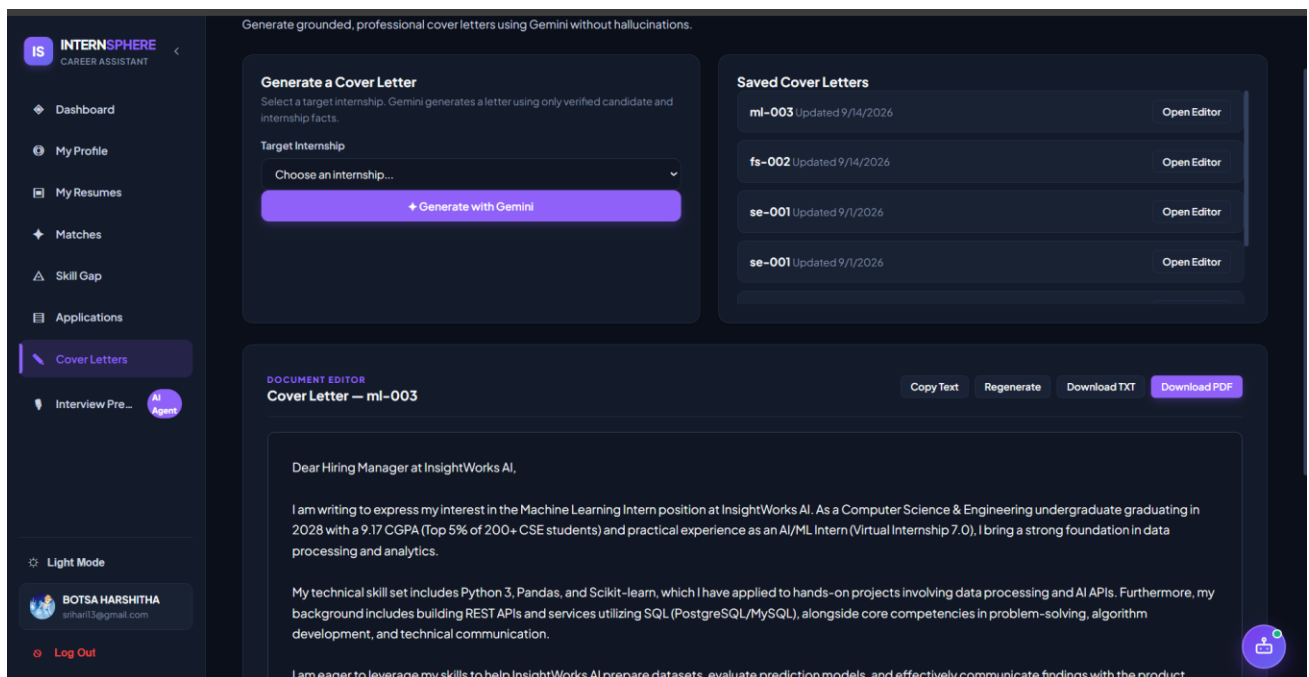


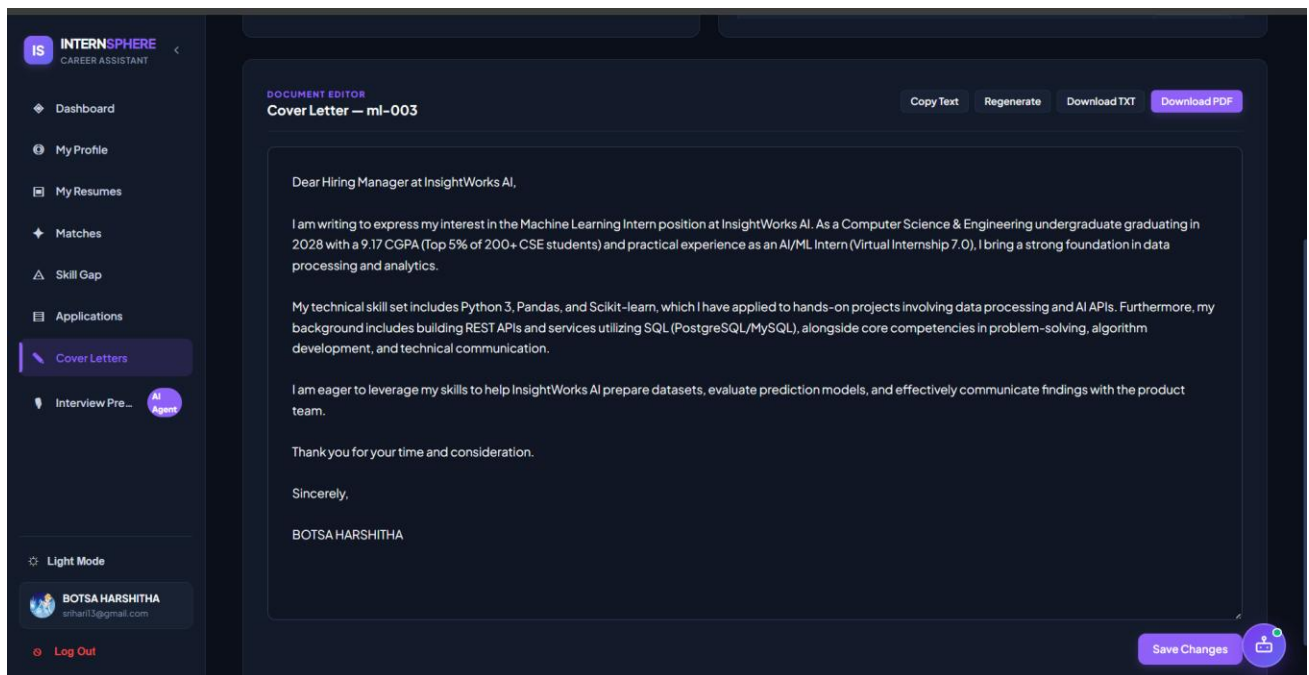
Feature 5: AI Cover Letter Generator & PDF Export

Description: Generates tailored, factual cover letters for specific internships with inline editing and instant PDF downloads.

Implementation: Prompt engineering instructs Gemini to use strictly candidate and job facts without hallucinating qualifications. The letter is rendered to a clean standard PDF-1.4 file via `services/pdf_generator.py`.

Output / Result:





Downloaded pdf

Dear Hiring Manager at InsightWorks AI,

I am writing to express my interest in the Machine Learning Intern position at InsightWorks AI. As a Computer Science & Engineering undergraduate graduating in 2028 with a 9.17 CGPA (Top 5% of 200+ CSE students) and practical experience as an AI/ML Intern (Virtual Internship 7.0), I bring a strong foundation in data processing and analytics.

My technical skill set includes Python 3, Pandas, and Scikit-learn, which I have applied to hands-on projects involving data processing and AI APIs. Furthermore, my background includes building REST APIs and services utilizing SQL (PostgreSQL/MySQL), alongside core competencies in problem-solving, algorithm development, and technical communication.

I am eager to leverage my skills to help InsightWorks AI prepare datasets, evaluate prediction models, and effectively communicate findings with the product team.

Thank you for your time and consideration.

Sincerely,

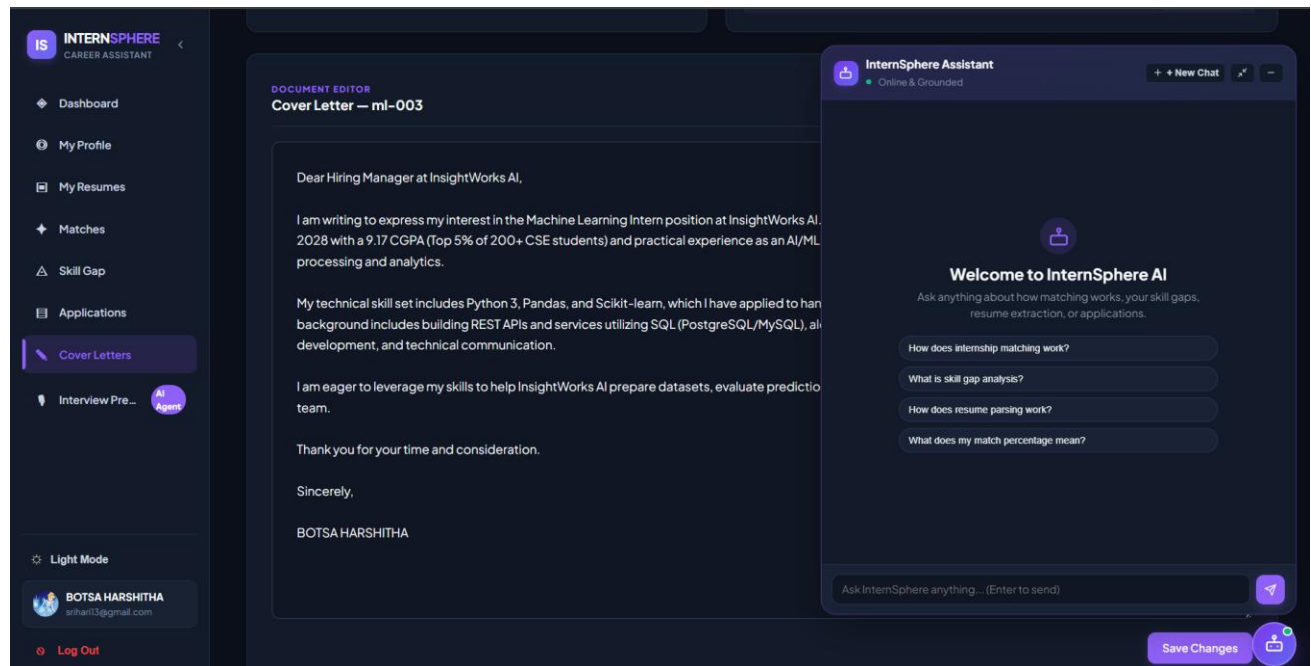
BOTSA HARSHITHA

Feature 6: Grounded Product AI Assistant

Description: Real-time interactive assistant answering user queries regarding platform navigation, matching criteria, and application rules.

Implementation: Implements RAG over data/product_knowledge.json with multi-turn session persistence in chat_sessions and chat_messages.

Output / Result:



Feature 7: AI Interview Preparation Coach with Voice Synthesis

Description: Resume-tailored interview coach delivering role recommendations, technical/behavioral questions, STAR answer advice, and roadmaps.

Implementation: Leverages candidate resume entities to formulate structured questions. Implements conversational memory, out-of-scope filters, and triggers the browser's Web Speech API (window.speechSynthesis) for read-aloud audio narration.

OPTIONAL:Voice

Output / Result:

IS

INTERNSPHERE

CAREER ASSISTANT

Dashboard

My Profile

My Resumes

Matches

Skill Gap

Applications

Cover Letters

Interview Pre...

AI Agent

Light Mode

BOTSARSHITHA

srihar15@gmail.com

Log Out

Supports PDF and DOCX (up to 10 MB)

Upload & Index with RAG

+ New Chat

PREVIOUS CHATS

What are my strongest skill...

How should I answer this int...

What are my strongest skills?

Target: Full-Stack Developer Intern

How should I answer this interview question: "In your project Sync Fission, you mentioned achieving sub-100ms query response times with zero downtime while exposing 20+ REST APIs using Node.js and PostgreSQL. Can you walk me through your indexing strategy and how you handled connection pooling or query optimization to maintain that latency under concurrent loads?" Provide an interview-ready answer.

Interview-Ready Response: Sync Fission Query Optimization & Architecture

Here is a structured, technically rigorous, interview-ready answer framed using the STAR method (Situation, Task, Action, Result) to help you clearly articulate your backend optimization strategy for Sync Fission.

Suggested Spoken Response

"When architecting Sync Fission, a real-time collaborative learning platform, maintaining high availability and low latency across 20+ REST APIs and WebSockets under concurrent load was a primary technical

Ask about your strongest skills, technical questions, answers, or documents... (Press Enter to send, Shift+Enter for new line)

Send

Upload & Index with RAG

+ New Chat

PREVIOUS CHATS

What are my strongest skill...

How should I answer this int...

What are my strongest skills?

Target: Full-Stack Developer Intern

Key Technical Talking Points to Remember

If the interviewer follows up with deeper technical questions, keep these specifics in mind based on your project background:

8. **Connection Pooling Details:** Mention that connection pooling prevents resource exhaustion on the PostgreSQL server by reusing connections instead of constantly opening and closing them.

9. **Indexing Trade-offs:** Be ready to mention that while indexes drastically speed up read queries, they slightly increase write overhead because the index must be updated on INSERT, UPDATE, or DELETE. Therefore, indexes were applied strictly based on frequent query filters and sorting keys.

10. **Node.js Asynchronous Handling:** Highlight how non-blocking I/O in Node.js combined with efficient connection pooling allowed the backend to handle high-throughput concurrent requests without blocking the event loop.

Sources: Candidate Resume

Listen

Ask about your strongest skills, technical questions, answers, or documents... (Press Enter to send, Shift+Enter for new line)

Send

10. Testing

Test Strategy

Testing was conducted using Python's standard library unittest framework to validate business logic, RAG retrieval quality, scoring formulas, and API endpoint security.

Test Cases

Test ID	Test Case Description	Expected Result	Actual Result	Status
T001	Register user with valid email & password	User record created, status 201	User created in DB	Pass
T002	Register duplicate username or email	HTTP 409 Conflict returned	HTTP 409 with error envelope	Pass
T003	Login with valid credentials	Status 200, JWT access token returned	JWT token returned	Pass
T004	Login with incorrect password	HTTP 401 Unauthorized returned	HTTP 401 returned	Pass
T005	Update profile education and skills	Status 200, updated JSON returned	Profile records updated	Pass
T006	Upload valid PDF resume	File saved, text parsed, DB updated	Resume parsed & saved	Pass
T007	Upload invalid file type (.txt/.exe)	HTTP 415 Unsupported Media Type	HTTP 415 returned	Pass
T008	Ingest internship dataset	Vector embeddings generated & saved	Index saved to JSON	Pass
T009	Match candidate against backend roles	Python/FastAPI roles ranked top	Roles correctly prioritized	Pass
T010	Verify skill-gap missing skill output	Missing required skills listed	Accurately displayed	Pass
T011	Submit duplicate internship application	HTTP 409 Conflict returned	HTTP 409 returned	Pass
T012	Withdraw pending application	Status set to "Withdrawn", timestamp recorded	Withdrawn successfully	Pass
T013	Generate cover letter without credentials	Safe error message without API key leak	Credential redacted	Pass
T014	Download cover letter as PDF	Valid PDF-1.4 binary stream returned	Valid PDF rendered	Pass
T015	Product Assistant question answering	Grounded answer with source references	Answer returned	Pass
T016	Interview Prep role recommendations	Relevant roles returned for candidate	Accurate roles returned	Pass

Test ID	Test Case Description	Expected Result	Actual Result	Status
T017	Interview Prep out-of-scope query	Politely declines non-career trivia	Politely declined	Pass
T018	Multi-turn interview conversation isolation	Messages isolated to owning session	Sessions isolated	Pass

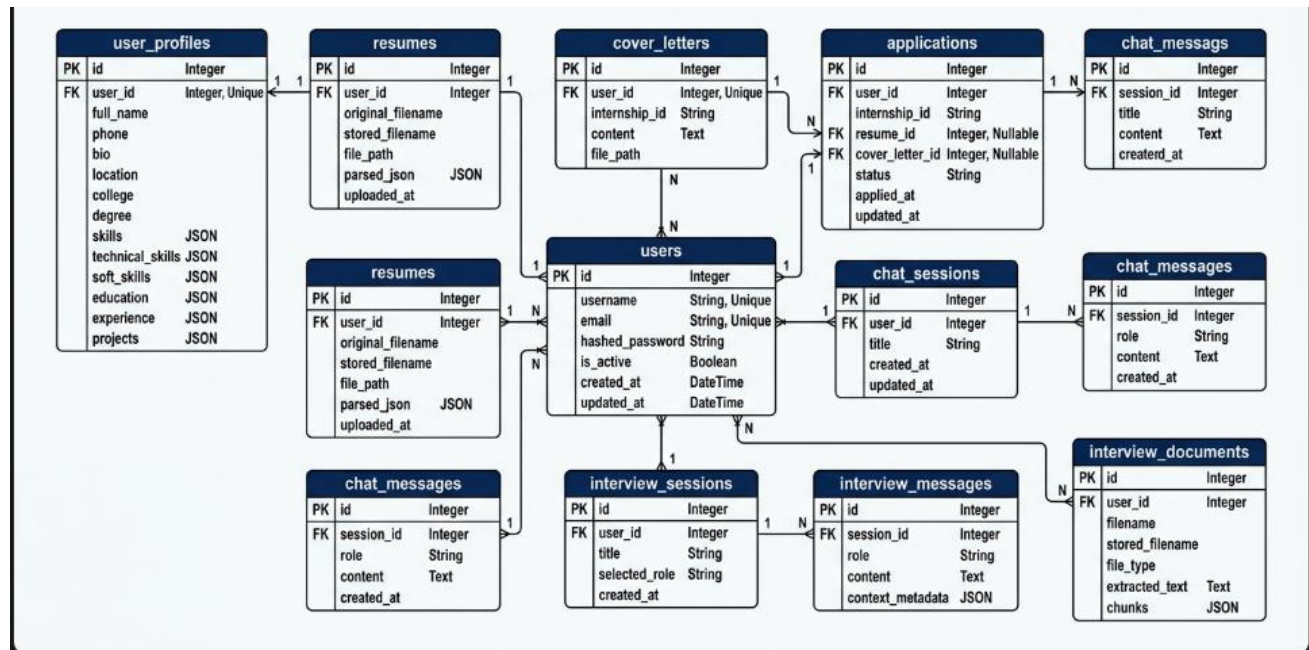
Testing Types Conducted

- **Functional Testing:** End-to-end user workflows (registration → upload → match → apply).
- **API & Schema Testing:** HTTP status codes, headers, and Pydantic validation across all endpoints.
- **Integration Testing:** Database persistence, cascade deletions, and relational foreign keys.
- **AI & RAG Verification:** Anti-hallucination verification, deterministic feature hashing, and semantic ranking stability.

11. Challenges & Solutions

Challenge Encountered	Technical Solution Implemented
Inconsistent Resume Formats (PDF/DOCX)	Built a dual-engine extractor using pdfplumber and python-docx coupled with deterministic regex for phone/emails and Gemini for nuanced skills.
Hallucination in Cover Letters	Designed strict zero-shot system prompts instructing Gemini to reference only explicit candidate facts and job requirements.
Offline Vector Search Without External Services	Developed a deterministic 384-dimensional feature hashing algorithm using SHA-256 and word n-grams, enabling vector search without external vector DBs.
Domain Separation Between AI Engines	Implemented regex pattern matching in interview_prep_service.py to intercept platform questions and route users to the Product Assistant.
Audio Playback Without Heavy Dependencies	Utilized the browser-native Web Speech API (window.speechSynthesis), avoiding heavy audio transcription libraries.
Safe Database Migrations in SQLite	Implemented apply_safe_schema_updates() to dynamically check table columns and append nullable fields without dropping user data.
Preventing Duplicate Applications	Enforced composite unique constraints (uq_application_user_internship) at the database level with clean HTTP 409 exception handling.

12.DATABASE SCHEMA



13. Final Project Status

Completed Features

- [Done] User Registration, Login, JWT Authentication, and Password Management
- [Done] User Profile Administration with Social Links, Academic Details, and Photo Uploads
- [Done] Authenticated PDF and DOCX Resume Parsing with Hybrid Regex + Gemini Extraction
- [Done] Persisted 384-Dimensional Vector Indexing with Offline Feature Hashing Support
- [Done] Explainable Semantic Internship Matching with Hybrid Scoring Formulas
- [Done] Granular Skill-Gap Analysis Highlighting Matching, Required, and Preferred Skills
- [Done] End-to-End Application Lifecycle Management with Status Tracking and Withdrawal
- [Done] AI-Powered Cover Letter Generator with In-Browser Editing and Binary PDF Export
- [Done] Grounded Product AI Assistant with Multi-Turn Session Memory and Documentation RAG
- [Done] AI Interview Preparation Coach with Role Advice, Questions, and STAR Evaluation
- [Done] Custom Study Guide Upload and Grounded Q&A for Interview Preparation
- [Done] Hands-Free Voice Synthesis via the Browser Web Speech API (Audio Listen)
- [Done] Automated Test Suite Covering Unit and Integration Scenarios

In Progress

- [In Progress] Advanced ATS Resume Optimization Scoring and Keyword Density Suggestions
- [In Progress] Export of Interview Prep Roadmaps into Calendar-Friendly Schedules

Pending / Future Scope

- [Pending] Integration with External Job Boards (LinkedIn, Indeed APIs) for Real-Time Openings
- [Pending] WebRTC-Based Interactive Mock Video and Audio Interviews with Speech-to-Text (Whisper)
- [Pending] Recruiter Dashboard for Posting Listings and Managing Candidate Submissions

Final Outcome Summary

The InternSphere platform has been successfully developed, integrated, and verified as a fully operational, end-to-end career acceleration solution. The application runs reliably on local environments using SQLite and is architected for instant deployment to PostgreSQL. All primary modules—including authentication, resume parsing, semantic matching, application tracking, cover letter drafting, product assistance, and interview preparation—function as specified.